Recommender Systems Tutorial

This hands-on tutorial explores the implementation of recommender systems using Python and the MovieLens dataset. Building on your knowledge of adaptive systems and user modeling, we'll implement practical recommendation algorithms that personalize content delivery based on user preferences and behavior patterns.

Introduction to Recommender Systems

Recommender systems are essential for adaptive interactive systems, guiding users to discover relevant items by predicting their preferences based on past behavior. In this tutorial, we will explore the fundamental concepts and practical implementation of these systems.

Collaborative Filtering

Collaborative filtering operates on the principle that users with similar tastes will like similar items. It recommends items by identifying patterns in the preferences of a community of users. For instance, if user A and user B have similar preferences for certain items, the system might recommend items liked by user A to user B, leveraging the collective intelligence of the user base.

Content-Based Filtering

Content-based filtering recommends items similar to those a user has previously shown interest in. This approach relies on analyzing the features of items (e.g., genre, director, keywords for movies) and constructing a profile of the user's preferences based on these characteristics. The system then matches this preference profile against the features of other available items to suggest new ones.

Installation of Required Libraries

Before we can begin working with the MovieLens dataset, we need to ensure that all the necessary Python libraries are installed on your system. These libraries provide the tools we will use for data manipulation, analysis, and visualization.

What you'll need to install

Open your terminal (on Mac/Linux) or command prompt (on Windows) and run the following command:

```
pip install numpy pandas scikit-learn matplotlib seaborn
```

This single command will install all five required libraries. If you are using Jupyter Notebook or Google Colab, you can run this command in a code cell by adding an exclamation mark at the beginning:

```
!pip install numpy pandas scikit-learn matplotlib seaborn
```

Importing the Libraries

Once installed, you'll need to import these libraries at the beginning of your Python script or notebook:

```
import numpy as np
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt
import seaborn as sns
```

Here is the role of each installed library:

- Numpy: Provides support for large arrays and matrices, as well as mathematical functions to operate on them.
- Pandas: Offers data structures like DataFrames for working with structured data (think of Excel spreadsheets in Python).
- Scikit-learn: Contains machine learning algorithms and tools, including the cosine-similarity function that we will use.
- Matplotlib: Creates visualizations such as graphs and diagrams.
- Seaborn: Built on Matplotlib, it facilitates the creation of attractive statistical graphics

Downloading and Loading the MovieLens Dataset

Now that our libraries are installed, we need to get the actual data we'll be working with. The MovieLens dataset is a collection of movie ratings provided by real users, maintained by the GroupLens research lab at the University of Minnesota.

Download the Dataset

1. Go to the official MovieLens website: **https://grouplens.org/datasets/movielens/**
2. Look for the "ml-latest-small.zip" file. This is a smaller version (around 100,000 ratings) that is perfect for learning.
3. Click to download the zip file to your computer.

Extract the Files

1. Locate the downloaded zip file (usually in your Downloads folder).
2. Extract/unzip the file. You should see a folder containing several CSV files.
3. The two files we need are:
   - `movies.csv` – contains movie information (movieId, title, genres)
   - `ratings.csv` – contains user ratings (userId, movieId, rating, timestamp)

Placing the files in your working directory

Move or copy `movies.csv` and `ratings.csv` into the same folder as your Python script or Jupyter Notebook.

Loading Data into Pandas

We can now use pandas to read these CSV files into DataFrames:

```
# Load the MovieLens dataset
movies = pd.read_csv('movies.csv')
ratings = pd.read_csv('ratings.csv')
```

Verify that the data has been loaded correctly

```
print("Movies DataFrame:")
print(movies.head())
print("\nRatings DataFrame:")
print(ratings.head())
```

You should see the first 5 rows of each dataset displayed. If you see an error like "`FileNotFoundError`", make sure the CSV files are in the correct directory.

Movies DataFrame

- `movieId`: A unique identifier for each movie
- `title`: The movie title, usually including the year
- `genres`: List of genres separated by vertical bars (e.g., "Action|Adventure|Sci-Fi")

Ratings DataFrame

- `userId`: A unique identifier for each user
- `movieId`: Link to the rated movie
- `rating`: The assigned rating (from 0.5 to 5.0 stars)
- `timestamp`: Time when the rating was given (in Unix timestamp format)

In-depth Analysis of Data

Using .info() to understand data types

The `.info()` method gives us a summary of the DataFrame's structure:

```
print("\n--- Movies Info ---")
movies.info()

print("\n--- Ratings Info ---")
ratings.info()
```

What to look for:

- `RangeIndex`: Total number of rows
- `Data columns`: Number of columns
- `Non-Null Count`: How many values are present (helps identify missing data)
- `Dtype`: Data type of each column

Using .describe() for a statistical summary

For numerical columns, `.describe()` provides statistics:

```
print("\n--- Ratings Statistics ---")
print(ratings.describe())
```

What you will learn:

- `Count`: Number of ratings
- `Mean`: Average rating (generally around 3.5)
- `Std`: Standard deviation (measures the dispersion of ratings)
- `Min/Max`: Lowest and highest ratings
- `25%, 50%, 75%`: Quartiles showing the distribution

Checking for Missing Values

Missing data can cause problems in our algorithms, so it's important to identify any gaps:

```
print("\n--- Missing Values in Movies ---")
print(movies.isnull().sum())

print("\n--- Missing Values in Ratings ---")
print(ratings.isnull().sum())
```

How to interpret the output:

- A value of `0` means this column has no missing data
- Any number greater than `0` indicates missing values that might need to be handled
- In the MovieLens dataset, you generally won't find missing values, which is good news!

Understanding Dataset Dimensions with .shape

The `.shape` attribute tells us the number of rows and columns in our DataFrames:

```
print(f"\nMovies DataFrame shape: {movies.shape}")
print(f"Ratings DataFrame shape: {ratings.shape}")
```

Example output:

- `(9742, 3)` for movies means: 9,742 movies and 3 columns
- `(100836, 4)` for ratings means: 100,836 ratings and 4 columns

This gives us an idea of the scale we are working with.

Counting Unique Users and Movies

It's helpful to know how many distinct users and movies we have:

```
print(f"\nNumber of unique users: {ratings['userId'].nunique()}")
print(f"Number of unique movies with ratings: {ratings['movieId'].nunique()}")
print(f"Total movies in catalog: {movies['movieId'].nunique()}")
```

What this tells us:

- How many different users have provided ratings
- How many movies have been rated at least once
- If there are any movies in the catalog that haven't been rated yet

This information is important for understanding data sparsity - most users rate only a small fraction of available movies, which is a key challenge in recommender systems.

Rating Distribution

This histogram shows how often each rating value appears:

```
plt.figure(figsize=(10, 6))
sns.countplot(x='rating', data=ratings, palette='viridis')
plt.title('Distribution of Movie Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of Ratings')
plt.show()
```

What to look for:

- You will likely see that ratings like 3.0, 4.0, and 5.0 are much more common
- Users tend to rate movies positively or neutrally
- Lower ratings (0.5, 1.0, 1.5) are generally less frequent
- This tells us that users are more likely to rate movies they enjoyed

Distribution of User Activity

This shows the number of ratings each user has submitted:

```
user_counts = ratings['userId'].value_counts()

plt.figure(figsize=(10, 6))
sns.histplot(user_counts, bins=30, kde=True, color='skyblue')
plt.title('Distribution of User Activity')
plt.xlabel('Number of Ratings per User')
plt.ylabel('Number of Users')
plt.yscale('log')  # Log scale helps visualize the wide range
plt.show()
```

What this reveals:

- The distribution is typically asymmetric
- Many users provide only a few ratings
- A smaller number of "power users" provide hundreds or even thousands of ratings
- The logarithmic scale (yscale='log') helps visualize this wide range
- This is important for understanding data sparsity

Distribution of Movie Popularity

```
movie_counts = ratings['movieId'].value_counts()

plt.figure(figsize=(10, 6))
sns.histplot(movie_counts, bins=30, kde=True, color='salmon')
plt.title('Distribution of Movie Popularity')
plt.xlabel('Number of Ratings per Movie')
plt.ylabel('Number of Movies')
plt.yscale('log')  # Log scale for better visualization
plt.show()
```

What this shows:

- Similar to user activity, movie popularity follows a skewed distribution
- A few blockbuster movies receive many ratings (hundreds or thousands)

- The vast majority of movies receive only a handful of ratings
- This is known as the "long tail" phenomenon

Why this is important for recommendation systems:

- Popular movies are easier to recommend (more data available)
- Obscure movies are harder to recommend (sparse data)
- A good recommendation system must balance popular and niche content
- This visualization helps us understand the challenge of data sparsity

Building a Simple Popularity-Based Recommender

Our first approach uses a popularity-based model that recommends movies based on aggregate statistics.

First, we need to **Calculate Statistics** for each movie. This involves computing the average rating and the total count of ratings to understand each movie's overall reception.

Next, to ensure statistical reliability, we will **Filter by Threshold**. This step removes movies that have received too few ratings, as their average ratings might not be representative of their true popularity.

Finally, we will **Rank Movies** by sorting the filtered movies based on their average rating in descending order. This allows us to identify the top-rated movies that have also received a significant number of reviews.

```
# Calculate movie statistics: mean rating and total count of ratings for each movie
movie_stats = ratings.groupby('movieId').agg({
    'rating': ['mean', 'count']
}).reset_index()

# Rename columns for clarity
movie_stats.columns = ['movieId', 'avg_rating', 'num_ratings']

# Set a minimum threshold for the number of ratings (e.g., 50 ratings)
min_ratings = 50

# Filter out movies that have received fewer ratings than the threshold
popular_movies = movie_stats[movie_stats['num_ratings'] >= min_ratings]

# Sort movies by average rating in descending order and select the top 10
top_movies = popular_movies.sort_values('avg_rating', ascending=False).head(10)

# Print the resulting top 10 popular movies
print(top_movies)
```

Output Example

```
     movieId  avg_rating  num_ratings
535      318    4.429022          317
693      858    4.289062          476
734     1196    4.215640          408
```

The movie with `movieId 318` has an average rating of 4.429 and has been rated 317 times, making it an excellent candidate for a popularity-based recommendation.

Implementing Collaborative Filtering with User-Item Matrix

1. Build User-Item Matrix

First, we need to transform our raw ratings data into a structured format called a **User-Item Matrix**. This matrix will have each user as a row, each movie as a column, and the cells will contain the rating a specific user gave to a specific movie. This matrix is often "sparse" because most users only rate a small fraction of all available movies.

We use a `pivot_table` for this. Imagine taking a flat table of 'user, movie, rating' and "pivoting" it around the user and movie IDs to create a grid. If a user hasn't rated a movie, that cell would normally be empty (NaN - Not a Number). To simplify calculations later, we fill these empty cells with `0`, which signifies that the user has not rated that movie or has no preference.

```
# Create user-item matrix
# ratings.pivot_table: This function reorganizes data.
#   - index='userId': Each unique userId becomes a row in the new matrix.
#   - columns='movieId': Each unique movieId becomes a column in the new matrix.
#   - values='rating': The values in the cells of the matrix will be the ratings.
#   For example, the cell at [User A, Movie X] will contain User A's rating for Movie X.
user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0) # .fillna(0): After pivoting, movies a user hasn't rated will appear as 'NaN' (Not a Number).
            # We replace all these 'NaN' values with 0. This is important because most similarity
            # calculations (like cosine similarity) expect numerical values and treat 0 as
            # 'no rating' or 'no preference' rather than an actual low rating.
```

Example Output of `user_item_matrix` (a small snippet for illustration):

```
          movieId    1   2   3        4        5 ...  193581  193583  193585  193587  193609
userId
   1       4.0  0.0  4.0  0.0  0.0 ...    0.0     0.0     0.0     0.0     0.0
   2       0.0  0.0  0.0  0.0  0.0 ...    0.0     0.0     0.0     0.0     0.0
   3       0.0  0.0  0.0  0.0  0.0 ...    0.0     0.0     0.0     0.0     0.0
   4       0.0  0.0  0.0  0.0  0.0 ...    0.0     0.0     0.0     0.0     0.0
```

2. Calculate User Similarity

Next, we need to understand how "similar" users are to each other. We achieve this by comparing their rating patterns across all movies. If two users tend to rate the same movies similarly (e.g., both give high ratings to action movies and low ratings to romantic comedies), they are considered similar. We use **cosine similarity** for this purpose. Cosine similarity measures the angle between two vectors (in this case, each user's row of ratings in the user-item matrix). A similarity score of 1 means users are perfectly similar, 0 means no similarity, and intermediate values indicate varying degrees of similarity.

```
# Calculate cosine similarity between all users
# user_item_matrix.values: Extracts the numerical data from the matrix as a NumPy array.
# Each row represents a user's ratings across all movies.
# cosine_similarity(...): Compares every user's rating pattern with every other user's pattern.
# It returns a square matrix where:
#   - Rows and columns represent users
#.  - Each cell [i, j] contains the similarity score between user i and user j
#   - The diagonal (where i == j) will always be 1.0 because a user is perfectly similar to themselves
user_similarity = cosine_similarity(user_item_matrix.values)

# Convert to DataFrame for easier manipulation
# We convert the similarity matrix into a pandas DataFrame so we can easily look up
# similarity scores by userId instead of just numeric indices.
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,  # Row labels are userIds
    columns=user_item_matrix.index  # Column labels are also userIds
)
```

Example output of `user_similarity_df` **(a small snippet):**

```
userId      1       2       3       4       5
userId
1       1.000000 0.123456 0.234567 0.345678 0.456789
2       0.123456 1.000000 0.567890 0.678901 0.789012
3       0.234567 0.567890 1.000000 0.890123 0.901234
4       0.345678 0.678901 0.890123 1.000000 0.012345
5       0.456789 0.789012 0.901234 0.012345 1.000000
```

In this example, user 1 has a similarity score of 0.456789 with user 5, which means they have somewhat similar tastes. The diagonal values are all 1.0 because each user is perfectly similar to themselves.

3. Define Recommendation Function

We create a function that takes a specific user's ID and generates a list of recommended movies for them. The core idea is to find users who are similar to our target user, look at what movies those similar users liked that our target user hasn't seen yet, and then suggest those movies.

```
def get_collaborative_recommendations(user_id, n_recommendations=5):
    # 1. Find similar users
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:11]

    print(f"\n--- Top 10 Similar Users for User {user_id} ---")
    print(similar_users)

    # 2. Get ratings from these similar users
    similar_users_ratings = user_item_matrix.loc[similar_users.index]

    print(f"\n--- Ratings by Similar Users ---")
    print(similar_users_ratings.head(3))

    # 3. Calculate weighted average ratings for movies
    weighted_ratings = similar_users_ratings.multiply(similar_users, axis=0).sum()

    # 4. Remove movies the target user has already rated
    user_rated_movies = user_item_matrix.loc[user_id]
    weighted_ratings = weighted_ratings[user_rated_movies == 0]

    print(f"\n--- Weighted Average Ratings (after removing rated movies) ---")
    print(weighted_ratings.head())

    # 5. Sort and return top recommendations
    recommendations = weighted_ratings.sort_values(ascending=False).head(n_recommendations)

    return recommendations
```

Advantages. Provides highly personalized recommendations based on community preferences. Improves over time as more ratings accumulate.
Limitations. Faces data sparsity issues when users have rated few items, and cold start problems when new users or items enter the system with no rating history.

Content-Based Filtering Using Movie Features

This approach analyzes the intrinsic characteristics of items rather than relying on user community patterns. This method builds recommendations based on the features of movies themselves, such as genres, or directors. Unlike collaborative filtering, it doesn't need to know what other users like. Instead, it learns what *you* like about movies and finds new movies that share those characteristics.

Step 1: Prepare Movie Features

```
# Split the genres string into a list
# movies['genres'] contains strings like "Action|Adventure|Sci-Fi"
# We split them by '|' to get individual genres as a list
movies['genre_list'] = movies['genres'].str.split('|')

# Example of what this looks like:
# Before: "Action|Adventure|Sci-Fi"
# After: ['Action', 'Adventure', 'Sci-Fi']
```

Step 2: Vectorize Movie Genres

```
# Initialize the MultiLabelBinarizer
# This tool converts lists of labels into binary (0/1) vectors
mlb = MultiLabelBinarizer()

# Fit and transform the genre lists
# fit_transform() does two things:
# 1. Learns all unique genres across all movies
# 2. Converts each movie's genre list into a binary vector
genre_matrix = mlb.fit_transform(movies['genre_list'])

# Convert to DataFrame for easier manipulation
# Each column represents one genre, each row represents one movie
# A value of 1 means the movie has that genre, 0 means it doesn't
genre_df = pd.DataFrame(
    genre_matrix,
    columns=mlb.classes_,  # Column names are the genre names
    index=movies['movieId'] # Row labels are movieIds
    )
```

Step 3: Build User Profile

```
def get_user_profile(user_id, ratings_df, genre_df, threshold=4.0):
    """
    Creates a user profile vector based on their highly-rated movies.
    Parameters:
    - user_id: The ID of the user
    - ratings_df: DataFrame containing all ratings
    - genre_df: DataFrame with movie genre vectors
    - threshold: Minimum rating to consider (default 4.0)
    Returns a vector representing the user's genre preferences """

    # 1. Get all movies this user rated highly (>= threshold)
    # Filter ratings to only include this user's high ratings
    user_ratings = ratings_df[
        (ratings_df['userId'] == user_id) &
        (ratings_df['rating'] >= threshold)
    ]

    # 2. Get the movieIds of these highly-rated movies
    liked_movies = user_ratings['movieId'].values

    # 3. Get the genre vectors for these movies
    # We look up each liked movie in our genre_df
    # This gives us the binary genre vectors for movies the user liked
    liked_movie_features = genre_df.loc[liked_movies]

    # 4. Aggregate (sum) all these vectors to create user profile
    # If a user liked 3 Action movies and 2 Comedy movies,
    # the Action column will have value 3, Comedy will have value 2
    user_profile = liked_movie_features.sum(axis=0)

    # 5. Normalize the profile (optional but recommended)
    # This converts counts into proportions
    # Makes the profile independent of how many movies the user rated
    if user_profile.sum() > 0:
        user_profile = user_profile / user_profile.sum()

        return user_profile
```

Example User Profile:

```
Action       0.25
Adventure    0.15
Comedy       0.30
Drama        0.20
Sci-Fi       0.10
```

This shows the user has a preference for Comedy (30%), followed by Action (25%).

Step 4: Compute Similarity Between User Profile and Movies

Now we calculate how similar each movie is to the user's profile:

```
def get_content_based_recommendations(user_id, ratings_df, genre_df, movies_df, n=10):
    """
    Generate content-based recommendations for a user.
    Parameters:
    - user_id: The ID of the user
    - ratings_df: DataFrame containing all ratings
    - genre_df: DataFrame with movie genre vectors
    - movies_df: DataFrame with movie information
    - n: Number of recommendations to return
    """
    # 1. Get the user's profile vector
    user_profile = get_user_profile(user_id, ratings_df, genre_df)

    # 2. Reshape user profile for cosine_similarity
    # cosine_similarity expects a 2D array, so we reshape from (20,) to (1, 20)
    user_profile_reshaped = user_profile.values.reshape(1, -1)

    # 3. Calculate similarity between user profile and ALL movies
    # This compares the user's preference vector with each movie's genre vector
    # Returns an array of similarity scores, one for each movie
    similarities = cosine_similarity(user_profile_reshaped, genre_df)

    # 4. Convert to a Series for easier manipulation
    # similarities[0] extracts the array from the 2D result
    # We create a Series with movieIds as index and similarity scores as values
    similarity_scores = pd.Series(
        similarities[0],
        index=genre_df.index
    )
```

```
    # 5. Remove movies the user has already rated
    # Get list of movies this user has already seen
    rated_movies = ratings_df[ratings_df['userId'] == user_id]['movieId'].values
    # Drop these from our similarity scores
    similarity_scores = similarity_scores.drop(rated_movies, errors='ignore')

    # 6. Sort by similarity score (highest first) and get top N
    top_recommendations = similarity_scores.sort_values(ascending=False).head(n)

    # 7. Merge with movie information to get titles
    recommended_movies = movies_df[
        movies_df['movieId'].isin(top_recommendations.index)
    ].copy()
    recommended_movies['similarity_score'] =
   recommended_movies['movieId'].map(top_recommendations)

    return recommended_movies.sort_values('similarity_score', ascending=False)
```

Step 5: Test the Content-Based Recommender

Now let's test our content-based recommender with a specific user:

```
# Test with user ID 1
user_id = 1
recommendations = get_content_based_recommendations(
    user_id=user_id,
    ratings_df=ratings,
    genre_df=genre_df,
    movies_df=movies,
    n=10
)

print(f"\nContent-Based Recommendations for User {user_id}:")
print(recommendations[['movieId', 'title', 'genres', 'similarity_score']])
```

Example Output:

```
   movieId                    title              genres similarity_score
0     1234          The Matrix (1999)  Action|Sci-Fi|Thriller          0.892
1     5678        Inception (2010)  Action|Sci-Fi|Thriller          0.875
2     9012 Blade Runner 2049 (2017)  Action|Sci-Fi|Thriller          0.863
3     3456    Interstellar (2014)      Drama|Sci-Fi          0.841
4     7890  The Dark Knight (2008)  Action|Crime|Drama          0.829
```

Understanding the Results:

- **similarity_score**: Ranges from 0 to 1, where 1 means perfect match with user's preferences
- Movies with higher scores share more genres with the movies the user rated highly
- The recommender suggests movies the user hasn't seen yet that match their taste profile

Advantages. 1. Works with few initial ratings. 2. Easy to explain recommendations. 3. No other user data needed. Recommends unpopular, matching content.

Limitations. 1. Recommends only similar items. 2. Quality depends on extracted features. 3. Overspecialization: